LecAssist: An AI Based Agent System for Recording, Summarising and Managing University Lectures

FYP Code: 23F-64

Software Requirement Specification

BY

ASAD SALEEM, CMS. ID: 023-23-0213

SHOAIB AHMED, CMS. ID: 023-23-0231

ASGHAR ALI, CMS. ID: 023-23-0218



Supervised by: Dr. Saif Hassan _____

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCE AND INFORMATION TECHNOLOGY

SUKKUR IBA UNIVERSITY

NOVEMBER 2026

1. INTRODUCTION ..... ***ERROR! BOOKMARK NOT DEFINED.***

1.1. Purpose of Document ..... **Error! Bookmark not defined.**
1.2. Intended Audience ..... **Error! Bookmark not defined.**
1.3. Document Convention, abbreviations, and acronyms ..... **Error! Bookmark not defined.**

2. OVERALL SYSTEM DESCRIPTION ..... ***ERROR! BOOKMARK NOT DEFINED.***

2.1. Project Background ..... **Error! Bookmark not defined.**
2.2. Project Scope ..... **Error! Bookmark not defined.**
2.3. Not in Scope ..... **Error! Bookmark not defined.**
2.4. Project Objectives ..... **Error! Bookmark not defined.**
2.5. Stakeholders ..... **Error! Bookmark not defined.**
2.6. Operating Environment ..... **Error! Bookmark not defined.**
2.7. System Constraints ..... **Error! Bookmark not defined.**
2.8. Assumptions & Dependencies ..... **Error! Bookmark not defined.**

3. EXTERNAL INTERFACE REQUIREMENTS ..... ***ERROR! BOOKMARK NOT DEFINED.***

3.1. Hardware Interfaces ..... **Error! Bookmark not defined.**
3.2. Software Interfaces ..... **Error! Bookmark not defined.**
3.3. Communications Interfaces ..... **Error! Bookmark not defined.**

4. FUNCTIONAL REQUIREMENTS ..... ***ERROR! BOOKMARK NOT DEFINED.***

4.1. Functional Hierarchy ..... **Error! Bookmark not defined.**
4.2. Use Cases ..... **Error! Bookmark not defined.**
4.2.1. [Title of use case] ..... **Error! Bookmark not defined.**

5. NON-FUNCTIONAL REQUIREMENTS ..... ***ERROR! BOOKMARK NOT DEFINED.***

5.1. Performance Requirements ..... **Error! Bookmark not defined.**
5.2. Safety Requirements ..... **Error! Bookmark not defined.**
5.3. Security Requirements ..... **Error! Bookmark not defined.**
5.4. User Documentation ..... **Error! Bookmark not defined.**

6. REFERENCES ..... ***ERROR! BOOKMARK NOT DEFINED.***

7. APPENDICES ..... ***ERROR! BOOKMARK NOT DEFINED.***

Introduction

Purpose of Document

This document states what the LecAssist system must do. It turns the idea described in the project proposal into requirements that can be built and tested. It describes the background of the problem, the boundary of the project, the people who will use the system, the environment the system will run in, every function the system must provide, the use cases that show how a teacher works with it, and the non functional requirements such as speed, safety and security.

The document is written for the first semester of the final year project. It is the agreement between the group and the supervisor about what will be delivered. The design of the system, the diagrams and the test cases are given in the companion Software Design Specification (SDS).

Intended Audience

- Project supervisor, Dr. Saif Hassan, who reviews and approves the requirements.
- The final year project evaluation committee of the Department of Computer Science, Sukkur IBA University.
- The three group members, who use this document as the work list for implementation and testing.
- Teachers of the department who will give requirements, provide lecture recordings and later use the system in a real class.
- The university IT staff who control the Moodle eLearning portal, because the system needs a teacher account, a web service token and a test course.

Document Convention, abbreviations, and acronyms

Main headings use the h1 style and sub headings use the H2 style of the department template. Body text is written in the normal paragraph style of the template. File names, commands, function names and model names are written exactly as they appear in the code, for example faster-whisper, pyannote.audio and add_assignment. Requirement identifiers are written as FR-x for functional requirements and NFR-x for non functional requirements. Use case identifiers are written as UC-xx.

Term	Meaning
ASR	Automatic Speech Recognition, the conversion of speech into text.
WER	Word Error Rate, the standard measure of transcription accuracy.
Diarization	Separating a recording into turns and labelling which speaker spoke each turn.
Code switching	Changing language inside one sentence, for example Urdu and English in the same line.
LLM	Large Language Model, the model that writes the summary and drives the agents.
Agent	A language model that is given tools and performs an action, not only text.
RAG	Retrieval Augmented Generation, answering a question from stored documents.
LMS	Learning Management System, used here for marks submission.
Moodle	The eLearning portal used by Sukkur IBA University at elearning.iba-suk.edu.pk.
Ollama	A tool that runs language models locally on the teacher's own machine.
OpenClaw	An open source, self hosted agent gateway that connects one agent to messaging channels such as Telegram and WhatsApp.
Playwright	A browser automation library used to operate the Moodle pages that have no web service.
FastAPI	The Python framework used for the local service inside the desktop application.
SQLite	The local file based database used for lectures, transcripts and summaries.
Vector index	A store of text embeddings (Chroma or FAISS) used for meaning based search.
OCR	Optical Character Recognition, reading text from a photograph of the whiteboard.
SRS / SDS	Software Requirements Specification / Software Design Specification.

Overall System Description

Project Background

A lecture does not end when the professor leaves the room. Notes still have to be written for the students who were absent. The assignment announced out loud has to be recorded somewhere and its deadline tracked. Supporting material has to be placed on the eLearning portal, and marks eventually have to be entered into the LMS. The work is repetitive, it is easy to postpone, and it eats hours that a teacher would rather spend teaching.
Converting speech into text is no longer the difficult part of this problem. Whisper gives word error rates close to human level on clean English audio, and speaker diarization has good open source implementations. Products such as Otter.ai, Notta, Fireflies and the meeting summary features inside Zoom and Microsoft Teams already give a transcript and a short summary, and some universities use lecture capture systems such as Panopto and Echo360. None of them solves the problem a teacher actually has, and the same five pieces are missing every time:

- Language. Teaching in Pakistani universities moves between Urdu and English inside one sentence. Speech models trained on a single language go weak on this kind of speech, and nothing built on a wrong transcript can be right.
- Action. Existing tools stop once they have produced a file. None of them opens the university eLearning portal, attaches the assignment that was announced in class, submits it, and then enters marks in the LMS.
- Memory of the whole semester. Meeting tools treat every recording as a separate event, but a teacher's question is usually about the whole course.
- Delivery. Students in this country are reached on WhatsApp. A summary that stays inside the application has not really been delivered.
- Privacy and cost. Cloud transcription needs classroom audio to be uploaded to another company's server, which many institutions will not allow, and per minute pricing becomes heavy over a full teaching load.

LecAssist is proposed as the system that closes all five gaps at once: it writes an accurate transcript of mixed Urdu and English classroom speech, understands both the academic content and the instructions inside it, performs the portal and LMS work for the teacher, delivers the result on the channels students already use, keeps the whole semester as a searchable record, and can do all of this without sending any audio outside the university network.

Project Scope

LecAssist is a desktop application for a university teacher. The teacher presses record at the start of the class and the system takes over from there. The boundary of the project is the work that follows one lecture, from the moment the recording starts to the moment the notes reach the students and the portal work is done. The system will provide:

- Recording of classroom audio with one button, with pause and resume in the same file, and import of audio recorded on another device.
- Noise cleaning, local transcription of mixed Urdu and English speech, speaker separation between the teacher and the students, time linked transcript lines, and permanent hand correction of any line.
- An academic summary in more than one style, a lecture title taken from the content, a key term list, highlights, and the assignments announced in class with their deadlines and submission methods.
- Automatic filing of every lecture under the correct subject and section using the uploaded timetable, and a coverage report that compares the uploaded course outline with the topics actually taught.
- An agent that performs the outside work: posting an approved assignment on Moodle, uploading supporting material, uploading a named file on command, and submitting marks to the LMS, always behind an approval step.
- Delivery of the summary to the class WhatsApp group and to the teacher's own Telegram or WhatsApp session, in which a reply can start a new action, and email sending from the application.
- Indexing of the whole semester so that a question asked in plain language is answered from all lectures with a reference to the lecture and date, with single lecture chat, comparison of two lectures and plain keyword search.
- Generation of multiple choice and short questions from a lecture, and of a complete exam paper from the whole semester, with export to PDF and Word.
- An archive with attachments, labels, a semester calendar, export, separate logins for several teachers, local backup, and one settings page for the provider, the microphone and the linked accounts.
- A log monitoring agent that reads the logs of the other agents and reports failed actions, repeated actions, unusual access and steps that take too much time or too many tokens.
- A fully offline mode in which speech recognition and the language model both run on the teacher's own machine and no audio or text leaves the device.

Not in Scope

- There is no student application. Students receive notes on the class group; they do not log in to LecAssist.
- There is no live captioning during the class. Processing starts after the recording is stopped.
- Video is not recorded or analysed. Only audio, whiteboard photographs and shared files are handled.
- Attendance is not marked, and no face or voice identification of individual students is performed. Speakers are labelled only as the teacher or a student.
- Answer scripts are not graded. The system submits marks that the teacher has already decided; it does not decide them.
- Languages other than Urdu and English are not targeted. Sindhi and other local languages are left for future work.
- Only Moodle is supported as the eLearning portal and LMS. Other platforms such as Canvas or Blackboard are not in this project.
- The system is a desktop application for Windows. A mobile or web version is not in scope; the phone is used only as a chat channel.
- Plagiarism checking, timetable generation and any financial or fee related work are not part of the system.

Project Objectives

1. To record and process classroom audio in real conditions, including background noise, pauses, interruptions and files recorded on other devices.
2. To produce accurate transcripts of mixed Urdu and English lecture speech, with the teacher's speech separated from the students', and with every line linked to its position in the audio.
3. To convert a transcript into useful academic output: a summary in more than one style, a lecture title, a list of key terms with meanings, important moments, and the assignments announced in class with their deadlines and submission methods.
4. To organise the archive from the teacher's timetable, so that each lecture is saved under the correct subject and section, and to compare the delivered lectures with the uploaded course outline so that the remaining topics are visible.
5. To perform the office work that follows a lecture, that is uploading assignments and material to the Moodle eLearning portal and submitting marks to the LMS, with approval of the teacher required before any outside action.
6. To deliver the output to the students and to the teacher on Telegram and WhatsApp through a chat session that accepts replies and new instructions.
7. To index the full semester so that questions can be answered from all lectures with references, and so that quizzes and complete exam papers can be generated from the stored material.
8. To allow the complete system to run offline on the teacher's own machine, and to allow a paid model from Anthropic or OpenAI to be selected instead when better quality is needed.

Stakeholders

Stakeholder	Interest in the system
University teacher (primary user)	Records the lecture, corrects the transcript, approves every outside action, and receives the time saving.
Students of the section	Receive the summary, the key terms and the assignment on the class group. They do not operate the system.
Project supervisor, Dr. Saif Hassan	Approves the requirements and the design, arranges access to teachers and to the portal, and evaluates the work.
Department of Computer Science and the FYP committee	Evaluate the project against the proposal and the documents.
University IT and Moodle administrator	Provides the teacher account, the web service token and a test course, and is concerned with the safety of the live portal.
Asad Saleem (developer)	Model provider layer, summary and extraction agents, portal and LMS agents, question answering, assessment and log monitoring.

Shoaib Ahmed (developer)	Electron and React desktop application, FastAPI service, database and vector index, screens and export.
Asghar Ali (developer)	Audio capture and cleaning, transcription and diarization testing, OCR, and measurement of word error rate and speaker error.

Operating Environment

- Hardware: a teacher's laptop with Windows 10 or 11, at least 16 GB of memory and about 20 GB of free disk space for a semester of recordings. An NVIDIA GPU is optional and only makes transcription faster.
- Audio input: the built in laptop microphone, or a USB or lapel microphone, captured on Windows through the WASAPI interface.
- Desktop application: Electron with React and TypeScript, running a local Python 3.11 or newer service built with FastAPI on the loopback address 127.0.0.1.
- Storage: a local SQLite database file with a vector index (Chroma or FAISS) next to it, and a folder of audio files and attachments on the same machine.
- Speech models: faster-whisper running Whisper large-v3-turbo, and pyannote.audio speaker-diarization-3.1, both downloaded once and then run locally. FFmpeg must be installed and on PATH.
- Language model: a local model served by Ollama, or the Anthropic Claude API or the OpenAI API when the department chooses the paid option.
- External systems: the university Moodle site at elearning.iba-suk.edu.pk over HTTPS, a local Moodle instance used as the test copy during development, Telegram and WhatsApp through an OpenClaw gateway, and an SMTP server for email.
- Network: an internet connection is needed only for portal work, messaging and the paid model. In the fully local setting the system works with no internet at all.

System Constraints

Software constraints

- Speech recognition and diarization must run locally, so model size and speed are limited by the teacher's machine.
- pyannote.audio models are gated on Hugging Face and need an access token, which must be set before diarization can run.
- Moodle does not expose a web service for every action. Creating an assignment and uploading a file are done through browser automation, which depends on the page structure of the installed Moodle version.
- The application targets Windows, because that is what the teachers of the department use.

Hardware constraints

- The system must work on a normal laptop with 16 GB of memory and no GPU. On CPU, transcription is slower than real time, so processing happens after the class and not during it.
- Recording quality depends on one microphone placed near the teacher. Students sitting far from it may be recorded weakly.

Cultural and language constraints

- Classroom speech mixes Urdu and English inside single sentences, and technical terms stay in English. The transcript must keep both scripts and must not force one language.
- Urdu is written right to left, so the transcript editor and every exported document must display mixed direction text correctly.

Legal and privacy constraints

- Classroom audio contains the voices of students. The teacher must announce the recording to the class, the audio stays on the teacher's machine, and only the summary is shared.
- Automated messaging on WhatsApp outside the official Business API is against the WhatsApp terms of service, so Telegram is the primary channel and WhatsApp is optional.
- Moodle credentials and tokens belong to the teacher. They must be stored on the teacher's machine only and must never be placed in the code or in the repository.

Environmental constraints

- A real classroom has fan noise, chairs, side conversation and echo. Audio cleaning is a required stage and not an extra feature.

User constraints

- Teachers are not technical users and have very little time. The main screen must work with one button, and the system must choose the subject and section by itself.
- No outside action may surprise the teacher. The system shows what it is going to do and waits for approval.

Off the shelf component constraints

- Whisper, pyannote.audio, FFmpeg, Playwright, Ollama and OpenClaw are open source components with their own licences and their own release changes, which the project has to follow.
- Paid APIs from Anthropic and OpenAI have per token cost and rate limits, which is one reason the local option exists.

Assumptions & Dependencies

Assumptions

- The teacher records the whole class on one machine and that machine stays on until processing is finished.
- The timetable and the course outline are uploaded once at the start of the term and are correct.
- The teacher reviews the summary and the detected assignment before anything is sent or uploaded.
- Class WhatsApp groups already exist and the teacher is a member of them.
- Each teacher uses the system for their own subjects, so one installation serves a small number of users and not the whole university.

Dependencies

- Real lecture recordings from teachers of the department, which every later test depends on.
- A Moodle teacher account, a web service token and a test course, arranged through the supervisor. Until these are granted, the agent is developed against a local Moodle instance.
- Hugging Face access to the pyannote models, and a working FFmpeg installation.
- Availability of Ollama and of a paid API key when the paid provider is selected.
- An OpenClaw gateway for Telegram and WhatsApp delivery.

External Interface Requirements

LecAssist sits between the teacher and four groups of outside parts: the microphone and audio files, the local speech and language models, the Moodle portal and LMS, and the messaging and email channels. The context diagram below shows these interfaces at a high level.

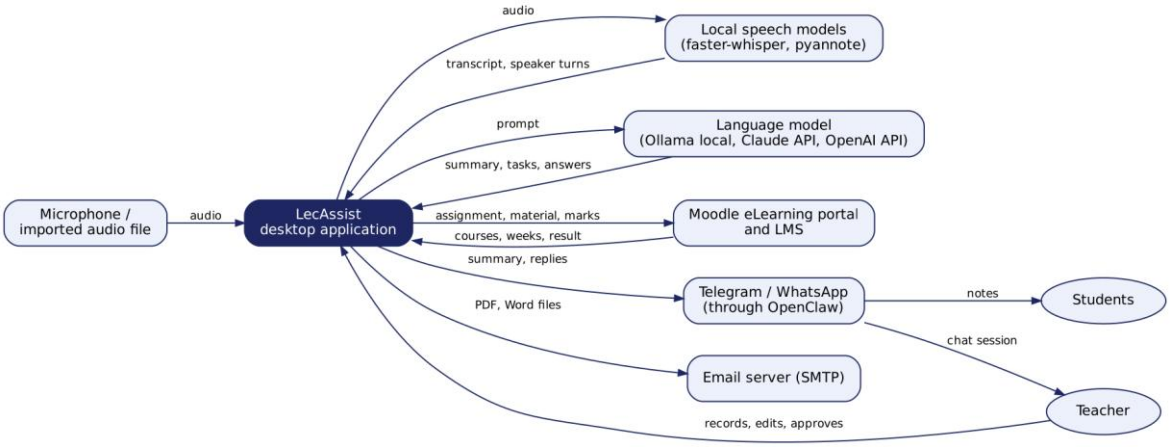


Figure 1: Context diagram of LecAssist

Hardware Interfaces

- Microphone interface. The system captures audio from the selected input device through the Windows WASAPI interface using the sounddevice library, at 16 kHz mono, which is the format the speech models expect. The teacher can choose the device from the settings page, and the level is shown while recording.

- Storage interface. Audio files, the SQLite database, the vector index and exported documents are written to the local disk of the teacher's machine. A semester of compressed audio is expected to stay under 20 GB.
- Optional GPU interface. If an NVIDIA GPU with CUDA is available, transcription runs on it with float16 precision; otherwise it runs on the CPU with int8 precision. The behaviour of the system does not change, only the speed.
- Imported device interface. Audio or video files recorded on a mobile phone, a camera or a recorder are read from the file system and converted with FFmpeg, so no direct device connection is required.

Software Interfaces

Component and version	Purpose and data exchanged
FFmpeg (current stable build)	Called as a command line process to convert any input file into 16 kHz mono PCM WAV. Input: a media file path. Output: a WAV file path.
faster-whisper with Whisper large-v3-turbo	Local transcription. Input: a WAV file, language code and an initial prompt written in mixed Urdu and English. Output: segments with start time, end time and text, saved as JSON.
pyannote.audio speaker-diarization-3.1	Speaker separation. Input: the same WAV file and a Hugging Face token. Output: speaker turns with start time, end time and a speaker label, saved as JSON.
noisereduce and soundfile	Optional noise cleaning before transcription. Input and output are WAV files.
jiwer	Word error rate measurement against a hand written reference transcript. Output: WER with substitution, deletion and insertion counts.
Ollama (local server on 127.0.0.1:11434)	Runs the local language model and supports tool calling and JSON schema output. Input: messages and tool definitions. Output: text, tool calls or a JSON object that matches a Pydantic schema.
Anthropic Claude API and OpenAI API	Optional paid providers behind the same internal interface as Ollama. Input: messages over HTTPS with an API key. Output: text or tool calls.
Moodle web service (REST, JSON)	Reading side of the portal work. Functions used: core_webservice_get_site_info, core_enrol_get_users_courses and core_course_get_contents, to find the course by short name and the week section by number. Input: a web service token and parameters. Output: JSON.
Playwright with Chromium	Writing side of the portal work, for the pages that have no web service: login, create assignment with due date and attachment, upload material and create a course. Input: course id, section number, name, due date and a file path.
SQLite 3	Local database for lectures, transcripts, summaries, assignments, deliveries and agent action logs.
Chroma or FAISS	Vector index of transcript chunks for meaning based search over the semester.
OpenClaw gateway	Keeps the Telegram and WhatsApp sessions open and passes messages between the agent and the chat channels.
Telegram Bot API	Sends the summary and receives replies over HTTPS using a bot token.
Tesseract OCR and PyMuPDF	Reads text from whiteboard photographs and from slide decks so that slides can be matched with the transcript.
python-docx and ReportLab	Export of summaries, quizzes and papers as Word and PDF files.

Communications Interfaces

- Internal communication. The Electron interface talks to the FastAPI service over HTTP on 127.0.0.1 only. The port is not opened to the network. Long jobs such as transcription report progress to the interface through a server sent event stream.
- Moodle communication. All calls go over HTTPS. Web service calls are REST with JSON responses and carry a token that belongs to the teacher. Browser automation uses the same HTTPS site through Chromium with the teacher's own login. The system never stores the password in plain text in the project files.
- Messaging. Telegram uses the Bot API over HTTPS with a bot token. WhatsApp is reached through the OpenClaw gateway. Messages are plain text with optional PDF attachments. Delivery status is written back into the database.
- Email. Outgoing mail uses SMTP over TLS on port 587 with the teacher's own account details.

- Paid model APIs. HTTPS requests with an API key held in the local settings store, sent only when the paid provider is selected.
- Data rates and limits. A fifty minute lecture produces about 50 MB of WAV audio and about 8000 words of transcript. Uploads to Moodle are single files, usually under 20 MB. No live streaming of audio to any outside service takes place at any time.

Functional Requirements

Functional Hierarchy

The system is divided into eleven modules. The figure below shows the modules and their sub functions, and the table after it lists the functional requirements of each module.

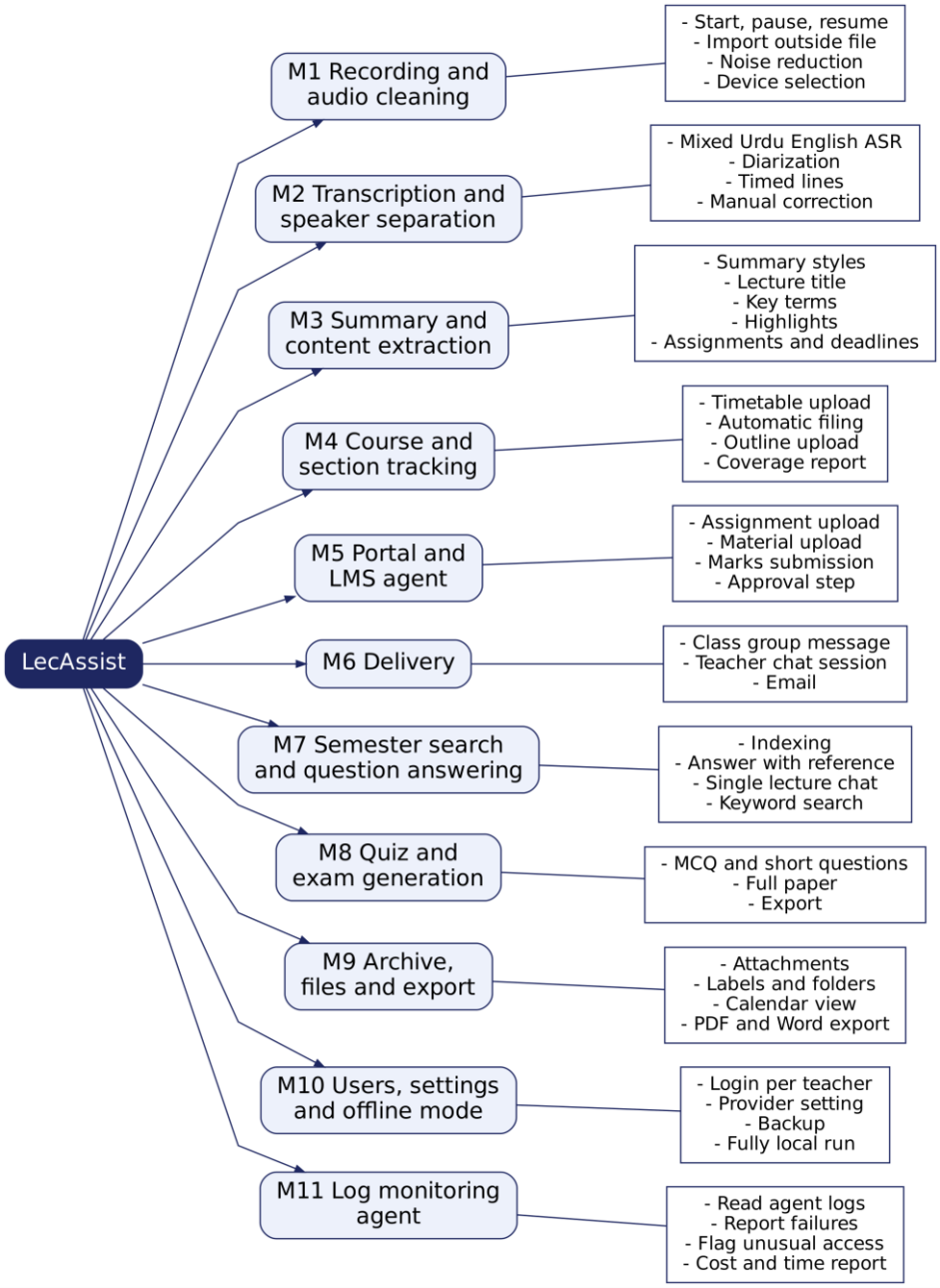


Figure 2: Functional hierarchy of LecAssist

ID	Module	Functional requirement
FR-1.1	M1 Recording	The system shall start, pause, resume and stop a recording with one button, writing one class into one file.

FR-1.2	M1 Recording	The system shall import an audio or video file recorded on another device and process it in the same way as a live recording.
FR-1.3	M1 Recording	The system shall convert any input to 16 kHz mono WAV and shall optionally reduce background noise before transcription.
FR-2.1	M2 Transcription	The system shall transcribe the lecture locally and shall keep mixed Urdu and English sentences as they were spoken.
FR-2.2	M2 Transcription	The system shall separate the teacher's speech from student speech and shall store class questions separately.
FR-2.3	M2 Transcription	The system shall store a start time and an end time for every transcript line and shall play the audio from that point when the line is clicked.
FR-2.4	M2 Transcription	The system shall let the teacher correct any transcript line, and the correction shall be used by every summary and search result made from that line.
FR-2.5	M2 Transcription	The system shall report word error rate against a hand written reference transcript when one is provided.
FR-3.1	M3 Summary	The system shall produce an academic summary of the lecture with greetings, attendance and side talk removed.
FR-3.2	M3 Summary	The system shall produce the summary in a short, a detailed and an exam focused style from the same transcript.
FR-3.3	M3 Summary	The system shall give every lecture a title taken from its own content.
FR-3.4	M3 Summary	The system shall list the new terms introduced in the lecture with a short meaning for each.
FR-3.5	M3 Summary	The system shall save a highlight when the teacher marks something as important, for example by saying it will come in the exam.
FR-3.6	M3 Summary	The system shall extract each announced assignment with its task, deadline and submission method and shall place the deadline on a calendar.
FR-3.7	M3 Summary	The system shall allow the teacher to edit the summary before it is sent or exported.
FR-4.1	M4 Course tracking	The system shall accept a timetable upload that gives subject, section, day and time slot.
FR-4.2	M4 Course tracking	The system shall file a finished recording under the subject and section that matches its own date and time, without asking the teacher to choose a folder.
FR-4.3	M4 Course tracking	The system shall keep a recording that matches no slot as an unscheduled class and shall let the teacher mark it as an extra or revision class.
FR-4.4	M4 Course tracking	The system shall keep separate lecture numbers and a separate calendar for every subject and section.
FR-4.5	M4 Course tracking	The system shall accept a course outline per subject and shall report, per section, which topics are covered, which were touched briefly and which remain.
FR-5.1	M5 Portal and LMS agent	The system shall post an approved assignment to the correct course and week on Moodle, with its due date and an optional attached file.
FR-5.2	M5 Portal and LMS agent	The system shall upload supporting material for the topic taught to the same course and week.
FR-5.3	M5 Portal and LMS agent	The system shall upload a named file to the portal when the teacher asks for it in plain words.
FR-5.4	M5 Portal and LMS agent	The system shall submit marks to the LMS from a prepared sheet, or by entering values one by one where bulk upload is not supported.
FR-5.5	M5 Portal and LMS agent	The system shall show every outside action before performing it and shall perform it only after the teacher approves.
FR-5.6	M5 Portal and LMS agent	The system shall record every attempted action, its result and any error in an agent action log.
FR-6.1	M6 Delivery	The system shall send the approved summary to the class group on WhatsApp or Telegram.
FR-6.2	M6 Delivery	The system shall keep an open chat session with the teacher in which a reply can ask a question or start a new action such as an upload.
FR-6.3	M6 Delivery	The system shall email a summary, quiz or paper from the application.
FR-7.1	M7 Semester search	The system shall index every lecture of the semester inside its own subject and section.

FR-7.2	M7 Semester search	The system shall answer a question asked in plain language from all lectures and shall give the lecture and date the answer came from.
FR-7.3	M7 Semester search	The system shall allow a question to be asked about one lecture alone, and shall allow two lectures to be compared.
FR-7.4	M7 Semester search	The system shall provide plain keyword search over all transcripts and summaries.
FR-8.1	M8 Assessment	The system shall generate multiple choice and short questions from a lecture, grounded in what was said in that class.
FR-8.2	M8 Assessment	The system shall generate a complete exam paper from the whole semester, balanced over weeks and difficulty.
FR-9.1	M9 Archive	The system shall attach whiteboard photographs and shared PDFs to a lecture and shall read text from them.
FR-9.2	M9 Archive	The system shall support labels, folders, lecture types, renaming and deleting, and a semester calendar view.
FR-9.3	M9 Archive	The system shall export any lecture, summary, quiz or paper to PDF or Word.
FR-10.1	M10 Users and settings	The system shall support several teachers on one installation with separate logins, archives and settings.
FR-10.2	M10 Users and settings	The system shall back up the archive locally, with cloud backup as an option chosen by the teacher.
FR-10.3	M10 Users and settings	The system shall let the teacher choose the model provider, the microphone, the linked accounts and the theme from one settings page.
FR-10.4	M10 Users and settings	The system shall run the complete main path with no audio or text leaving the device when the local provider is selected.
FR-11.1	M11 Log monitoring	The system shall read the logs of the other agents and shall produce a report after fixed intervals.
FR-11.2	M11 Log monitoring	The report shall list failed actions, actions repeated many times, unusual access and steps that take too much time or too many tokens.

Use Cases

The use case diagram below shows the twelve main use cases of the system and the actors that take part in them. The teacher is the only human actor who operates the application. The Moodle portal, the language model and the students are external actors. Each use case is described after the diagram.

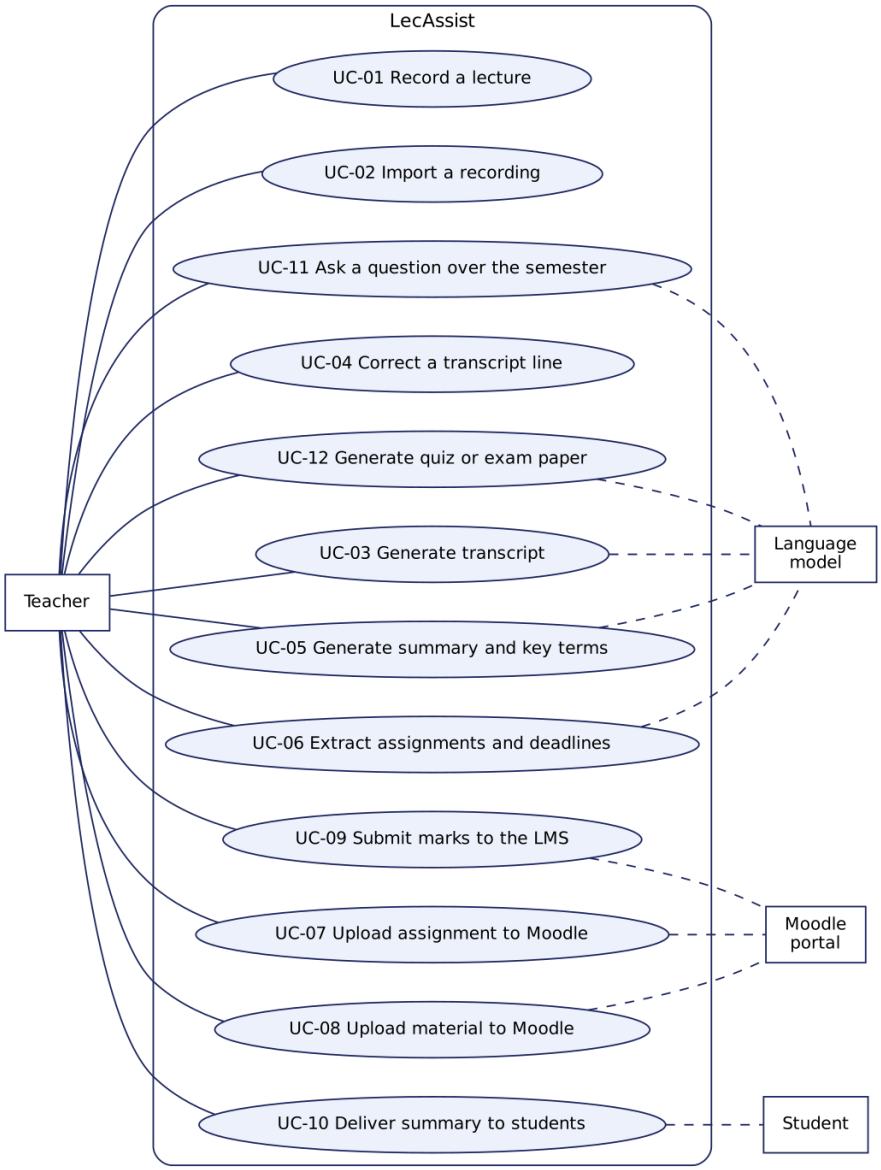


Figure 3: Use case diagram of LecAssist

UC-01 Record a lecture

UC-01: Record a lecture		
Use case Id:	UC-01	
Actors:	Teacher (initiates)	
Feature:	M1 Recording and audio cleaning	
Pre-condition:	The teacher is logged in, a microphone is connected and the timetable has been uploaded.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher opens the application and presses Start recording.	The system starts capturing audio from the selected microphone, shows the running time and writes the audio to disk continuously.

2	The teacher pauses during a break and presses resume afterwards.	The system continues writing into the same file so that one class stays one record.
3	The teacher presses Stop at the end of the class.	The system closes the file, reads its own date and time, matches them against the timetable and saves the lecture under the correct subject and section.
4	The teacher leaves the room.	The system starts the processing chain: audio cleaning, transcription, speaker separation and summary writing.
Alternate Scenarios		
1a: No microphone is found. The system shows the device list and asks the teacher to choose one.		
3a: The date and time match no timetable slot. The system saves the lecture as an unscheduled class and asks the teacher to mark it as an extra class, a revision class or to choose the subject.		
4a: The machine is closed before processing finishes. The system resumes the unfinished job when the application is started again.		
Post Conditions		
Step#	**Description**	
1	The recording is stored on disk and is listed under the correct subject and section.	
2	A processing job for that lecture has been queued.	
Use Case Cross referenced	UC-02, UC-03	

UC-02 Import an existing recording

UC-02: Import an existing recording		
Use case Id:	UC-02	
Actors:	Teacher (initiates)	
Feature:	M1 Recording and audio cleaning	
Pre-condition:	An audio or video file of a lecture exists on the machine.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher presses Import file and chooses the file.	The system converts it with FFmpeg to 16 kHz mono WAV.
2	The teacher selects the subject, section and date of that class.	The system creates a lecture record with those details.
3	The teacher confirms.	The system runs the same processing chain used for a live recording.
Alternate Scenarios		
1a: The file format cannot be read. The system reports that the file could not be converted and keeps the original untouched.		
Post Conditions		
Step#	**Description**	
1	The imported lecture appears in the archive with a transcript and a summary.	
Use Case Cross referenced	UC-01, UC-03	

UC-03 Generate the transcript with speakers

UC-03: Generate the transcript with speakers	
Use case Id:	UC-03
Actors:	Teacher, Language model (supporting)
Feature:	M2 Transcription and speaker separation

Pre-condition:	A recording has been stored for the lecture.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The processing job starts automatically after recording stops.	The system cleans the audio and reduces fan noise, background talking and microphone hiss.
2		The system transcribes the audio locally with the Whisper based model, keeping mixed Urdu and English sentences as spoken.
3		The system runs speaker diarization and labels each turn as the teacher or a student.
4		The system merges the transcript segments with the speaker turns and stores one timed line per segment.
5	The teacher opens the lecture.	The system shows the transcript with times and speaker labels, and plays the audio from a line when it is clicked.
Alternate Scenarios		
2a: The Hugging Face token for the diarization model is missing. The system still produces the transcript and marks all speakers as unknown, and warns the teacher.		
2b: The language is detected wrongly. The teacher can re-run transcription with the language fixed to Urdu, English or automatic.		
Post Conditions		
Step#	**Description**	
1	A timed transcript with speaker labels is stored for the lecture and is ready for summarisation.	
Use Case Cross referenced	UC-01, UC-04, UC-05	

UC-04 Correct a transcript line

UC-04: Correct a transcript line		
Use case Id:	UC-04	
Actors:	Teacher (initiates)	
Feature:	M2 Transcription and speaker separation	
Pre-condition:	A transcript exists for the lecture.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher clicks a line that looks wrong.	The system plays the audio from the time of that line.
2	The teacher edits the text or changes the speaker label and saves.	The system stores the corrected line and marks it as edited.
3	The teacher asks for the summary again.	The system regenerates the summary, key terms and extracted tasks using the corrected transcript.
Alternate Scenarios		
2a: The teacher cancels the edit. The original line is kept.		
Post Conditions		
Step#	**Description**	
1	The corrected line is permanent and is used by every summary, export and search result made afterwards.	

Use Case Cross referenced	UC-03, UC-05, UC-11

UC-05 Generate the summary, title and key terms

UC-05: Generate the summary, title and key terms		
Use case Id:	UC-05	
Actors:	Teacher, Language model (supporting)	
Feature:	M3 Summary and content extraction	
Pre-condition:	A transcript exists for the lecture and a model provider is configured.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher chooses a summary style, or accepts the default.	The system sends the transcript and the style to the selected model provider.
2		The system writes the academic summary, gives the lecture a title from its content, lists the new key terms with meanings and saves any highlight the teacher marked in class.
3	The teacher reads and edits the summary.	The system saves the edited version as the one that will be delivered and exported.
Alternate Scenarios		
1a: The local model is not running. The system reports that Ollama could not be reached and offers to switch to the paid provider for this lecture.		
2a: The summary is poor because the transcript is weak. The teacher corrects transcript lines (UC-04) and regenerates.		
Post Conditions		
Step#	**Description**	
1	A summary, a lecture title and a key term list are stored for the lecture.	
Use Case Cross referenced	UC-03, UC-04, UC-06, UC-10	

UC-06 Extract assignments and deadlines

UC-06: Extract assignments and deadlines		
Use case Id:	UC-06	
Actors:	Teacher, Language model (supporting)	
Feature:	M3 Summary and content extraction	
Pre-condition:	A transcript exists for the lecture.	
Scenarios		
Step#	**Action**	**Software Reaction**
1		The system reads the transcript and finds every announced task with its title, deadline and submission method, returned as a structured record.
2		The system shows the detected assignment next to the summary and puts the deadline on the subject calendar.
3	The teacher checks the detected assignment and corrects the date or the wording if needed.	The system saves the corrected assignment and keeps it ready for upload.
Alternate Scenarios		
1a: Nothing was announced in the class. The system reports that no assignment was found and no calendar entry is made.		

1b: A date was spoken loosely, for example next Friday. The system resolves it against the lecture date and shows the exact date for confirmation.	
Post Conditions	
Step#	**Description**
1	Zero or more assignments with deadlines are stored for the lecture and appear on the calendar.
Use Case Cross referenced	UC-05, UC-07

UC-07 Upload an assignment to the Moodle portal

UC-07: Upload an assignment to the Moodle portal		
Use case Id:	UC-07	
Actors:	Teacher (approves), Moodle portal (external)	
Feature:	M5 Agent for eLearning portal and LMS work	
Pre-condition:	An assignment has been detected or written, and the Moodle token and teacher login are configured.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher presses Approve upload on the detected assignment.	The system looks up the course by its short name and the week section by its number through the Moodle web service.
2		The system shows exactly what it is about to do: course, week, assignment name, due date and the file to attach.
3	The teacher confirms the action.	The agent logs in to Moodle in a browser, opens the new assignment page for that course and week, fills the name and due date, attaches the file and saves.
4		The system reports success, stores the result in the agent action log and marks the assignment as uploaded.
Alternate Scenarios		
1a: No course matches the short name. The system reports it and asks the teacher to pick the course from a list.		
1b: The course has no such week. The system reports it and asks for the correct week.		
3a: The teacher does not approve. Nothing is sent and the action is logged as cancelled.		
3b: Login fails or the page structure has changed. The system stops, reports the error, changes nothing on the portal and writes the failure to the log.		
Post Conditions		
Step#	**Description**	
1	The assignment exists on the Moodle course page under the correct week, or nothing was changed and the failure is recorded.	
Use Case Cross referenced	UC-06, UC-08, UC-09	

UC-08 Upload supporting material to the portal

UC-08: Upload supporting material to the portal	
Use case Id:	UC-08
Actors:	Teacher (approves), Moodle portal (external)
Feature:	M5 Agent for eLearning portal and LMS work
Pre-condition:	A lecture has been summarised and the teacher is logged in to the application.
Scenarios	

Step#	Action	Software Reaction
1	The teacher asks for material for the topic taught, or names a file to upload.	The system prepares the upload and shows the course, the week, the display name and the file.
2	The teacher approves.	The agent logs in, opens the add resource page for that course and week, uploads the file and saves.
3		The system reports the result and writes it to the agent action log.
Alternate Scenarios		
1a: The named file cannot be found on disk. The system reports it and asks for the correct path.		
2a: The teacher does not approve. Nothing is uploaded.		
Post Conditions		
Step#	**Description**	
1	The material appears on the Moodle course page in the correct week, or nothing was changed.	
Use Case Cross referenced	UC-07	

UC-09 Submit marks to the LMS

UC-09: Submit marks to the LMS		
Use case Id:	UC-09	
Actors:	Teacher (approves), Moodle LMS (external)	
Feature:	M5 Agent for eLearning portal and LMS work	
Pre-condition:	A marks sheet is ready, or the marks are given to the agent in the chat session.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher gives the marks sheet and names the course and the activity.	The system reads the sheet, matches the student identifiers with the course list and shows a preview of what will be entered.
2	The teacher approves.	The agent enters the marks, either by bulk upload or by filling the values one by one where bulk upload is not supported.
3		The system reports how many rows were entered, lists any row it could not match and writes the result to the log.
Alternate Scenarios		
1a: A student identifier does not match any enrolled student. That row is skipped and reported; the rest continue.		
2a: The teacher does not approve. No marks are entered.		
Post Conditions		
Step#	**Description**	
1	The marks are visible in the LMS grade book, and every skipped row is reported to the teacher.	
Use Case Cross referenced	UC-07	

UC-10 Deliver the summary to students and to the teacher

UC-10: Deliver the summary to students and to the teacher	
Use case Id:	UC-10
Actors:	Teacher (approves), Students (receive)
Feature:	M6 Delivery to students and to the teacher

Pre-condition:	A summary has been approved and a messaging channel is linked.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher presses Send to class group.	The system formats the summary and sends it to the class group through the linked channel.
2		The system also sends the summary to the teacher's own chat session and keeps that session open.
3	The teacher replies from the phone with a question or a new instruction.	The system answers from the lecture, or performs the instruction after showing it for approval.
4		The system records the delivery time and status for the lecture.
Alternate Scenarios		
1a: The channel is not linked. The system reports it and offers the settings page.		
1b: WhatsApp delivery fails. The system falls back to Telegram and tells the teacher which channel was used.		
Post Conditions		
Step#	**Description**	
1	The students have the summary on their class group and the teacher has an open chat session for that lecture.	
Use Case Cross referenced	UC-05, UC-11	

UC-11 Ask a question over the whole semester

UC-11: Ask a question over the whole semester		
Use case Id:	UC-11	
Actors:	Teacher (initiates), Language model (supporting)	
Feature:	M7 Semester search and question answering	
Pre-condition:	At least one lecture of the subject has been transcribed and indexed.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher types a question in plain language and chooses the scope: the whole semester, one lecture, or two lectures to compare.	The system searches the index of that subject and section and collects the most relevant transcript parts.
2		The system writes the answer from those parts and shows the lecture number, the date and the time position each part came from.
3	The teacher clicks a reference.	The system opens that lecture at that exact line and plays the audio from there.
Alternate Scenarios		
1a: Nothing relevant is found. The system says the answer is not in the stored lectures instead of inventing one.		
1b: The teacher remembers the exact word spoken. Plain keyword search is used instead of meaning based search.		
Post Conditions		
Step#	**Description**	
1	The teacher has an answer with references and can open the source lecture.	
Use Case Cross referenced	UC-03, UC-12	

UC-12 Generate a quiz or an exam paper

UC-12: Generate a quiz or an exam paper		
Use case Id:	UC-12	
Actors:	Teacher (initiates), Language model (supporting)	
Feature:	M8 Quiz and exam paper generation	
Pre-condition:	One lecture, or a set of lectures of the section, has been transcribed and indexed.	
Scenarios		
Step#	**Action**	**Software Reaction**
1	The teacher chooses a lecture or a range of weeks and the number and type of questions.	The system collects the stored material for that scope.
2		The system generates multiple choice and short questions grounded in what was said in those classes, with the answers, and balances them over weeks and difficulty.
3	The teacher edits or removes questions and presses export.	The system saves the paper and exports it as PDF or Word.
Alternate Scenarios		
1a: Too few lectures are stored for the chosen range. The system reports it and suggests a smaller scope.		
2a: A question is not grounded in the material. The teacher deletes it and asks for a replacement.		
Post Conditions		
Step#	**Description**	
1	A quiz or a complete exam paper is stored with the subject and can be printed or emailed.	
Use Case Cross referenced	UC-11	

Non-functional Requirements

Performance Requirements

ID	Requirement
NFR-1.1	Recording shall start within two seconds of the button press and shall not drop audio during a class of up to three hours.
NFR-1.2	On a laptop with 16 GB of memory and no GPU, a fifty minute lecture shall be transcribed and diarized within about the same length of time as the lecture itself, and faster when a CUDA GPU is available.
NFR-1.3	Word error rate on the mixed Urdu and English parts of the collected test recordings shall be at or below 25 percent, measured separately from the clean English parts. The summary stage is only trusted above that level after the teacher has corrected the transcript.
NFR-1.4	Summary, key term and assignment extraction for one lecture shall finish within five minutes on the local provider and within one minute on a paid API.
NFR-1.5	A question asked over the semester shall return an answer with references within fifteen seconds for a subject with up to thirty indexed lectures.
NFR-1.6	A portal action shall complete within two minutes, and shall be retried once automatically before it is reported as failed.
NFR-1.7	The archive shall hold at least one full semester per subject, that is about thirty lectures per section, without a visible slowdown in search or in the lecture list.
NFR-1.8	The application shall start and show the archive within ten seconds, and transcription shall run as a background job so that the interface stays usable.

Safety Requirements

ID	Requirement

NFR-2.1	No outside action, that is any upload, marks entry or message, shall be performed until the teacher has seen exactly what will be done and has approved it.
NFR-2.2	The agent shall never delete or overwrite existing content on the portal or in the LMS. It shall only add new items and fill empty fields.
NFR-2.3	Portal and LMS automation shall be developed and tested against a test copy of Moodle. It shall be pointed at a real course only after the actions are stable and the approval step is working.
NFR-2.4	Audio shall be written to disk continuously during recording, so that a crash or a power failure loses at most a few seconds and never the whole class.
NFR-2.5	An unfinished processing job shall be resumed when the application is started again, and a partly processed lecture shall never be shown as complete.
NFR-2.6	A clear indicator shall be visible on screen while recording is running, so that nobody in the room is recorded without the teacher knowing.
NFR-2.7	Every automated action, successful or failed, shall be written to the agent action log with its time, its parameters and its result, so that any wrong action can be traced and undone by hand.

Security Requirements

ID	Requirement
NFR-3.1	Each teacher shall have a separate login, a separate archive and separate settings on the same installation, and shall not be able to open another teacher's lectures.
NFR-3.2	In the fully local setting no audio, transcript or summary shall leave the machine. The teacher shall be able to see which provider is selected at all times.
NFR-3.3	Moodle passwords, web service tokens and API keys shall be stored in the operating system credential store or in a local configuration file outside the project folder. They shall never be written into the source code or into the Git repository.
NFR-3.4	The browser used for automation shall run with the teacher's own account and shall be closed after each action, leaving no open session behind.
NFR-3.5	The recording of a class shall be announced to the students by the teacher, audio shall stay on the teacher's machine, and only the summary shall be shared with the class.
NFR-3.6	A lecture, its audio and its transcript shall be deletable by the teacher at any time, and deletion shall remove the audio file, the database rows and the index entries together.
NFR-3.7	The log monitoring agent shall report unusual access, repeated failed logins and actions that were attempted outside the teacher's own courses.
NFR-3.8	Automated messaging shall use the official Telegram Bot API, and WhatsApp shall be used only through an account that the teacher has linked themselves, because automated messaging outside the official WhatsApp Business API is against the WhatsApp terms of service.

User Documentation

- A user manual in PDF with screenshots, covering first time setup, recording, correcting a transcript, approving an upload and linking a chat channel.
- A setup guide for the technical parts: installing FFmpeg, installing Ollama and pulling the local model, setting the Hugging Face token for diarization, and getting the Moodle web service token from the administrator.
- A README file in the project repository that explains the modules, the commands to run the pipeline and the environment variables that must be set.
- Short help text inside the application next to the settings that teachers are most likely to get wrong, such as the provider choice and the microphone choice.
- A demonstration video that shows one complete path from pressing record to the summary arriving on a phone and the assignment appearing on Moodle.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in Proc. 40th Int. Conf. on Machine Learning (ICML), Honolulu, HI, USA, 2023, pp. 28492–28518.

[2] H. Bredin, R. Yin, J. M. Coria, G. Gelly, P. Korshunov, M. Lavechin, D. Fustes, H. Titeux, W. Bouaziz, and M.-P. Gill, "pyannote.audio: Neural building blocks for speaker diarization," in Proc. IEEE ICASSP, Barcelona, Spain, 2020, pp. 7124–7128.

[3] S. Sitaram, K. R. Chandu, S. K. Rallabandi, and A. W. Black, "A survey of code-switched speech and language processing," arXiv:1904.00784, 2019.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. ICLR, Kigali, Rwanda, 2023.

[5] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[6] Moodle Pty Ltd, "Moodle web services and the core_course and core_enrol API," Moodle Developer Resources. [Online]. Available: https://moodledev.io/docs/apis/subsystems/external. [Accessed: 06-Nov-2026].

[7] Microsoft, "Playwright for Python documentation." [Online]. Available: https://playwright.dev/python/. [Accessed: 06-Nov-2026].

[8] Ollama, "Ollama: Get up and running with large language models locally." [Online]. Available: https://ollama.com/. [Accessed: 06-Nov-2026].

[9] OpenClaw, "OpenClaw: open source, self hosted agent gateway for messaging channels." [Online]. Available: https://openclaw.ai/. [Accessed: 06-Nov-2026].

[10] Anthropic, "Claude API documentation." [Online]. Available: https://docs.anthropic.com/. [Accessed: 06-Nov-2026].

[11] OpenAI, "OpenAI API documentation." [Online]. Available: https://platform.openai.com/docs/. [Accessed: 06-Nov-2026].

[12] LecAssist project repository. [Online]. Available: https://github.com/asadsehto/lecassist. [Accessed: 06-Nov-2026].

Appendices

Appendix A: Status of the implementation at the time of this document

About thirty percent of the system has been built and is in the project repository at https://github.com/asadsehto/lecassist. The parts listed below already run end to end on real lecture clips and on a local Moodle instance.

Part	What already works	Requirements covered
Audio preparation	Any audio or video file is converted with FFmpeg to 16 kHz mono WAV, with optional noise reduction saved as a separate file.	FR-1.2, FR-1.3
Transcription	faster-whisper with Whisper large-v3-turbo runs locally on CPU or CUDA, with Urdu as the default language and a mixed Urdu and English initial prompt, and writes timed segments as JSON.	FR-2.1, FR-2.3
Speaker separation	pyannote speaker-diarization-3.1 produces speaker turns, which are merged with the transcript segments by largest overlap into one timed, speaker labelled transcript in text and JSON.	FR-2.2, FR-2.3
Accuracy measurement	A reference transcript can be given on the command line and the pipeline reports word error rate with substitution, deletion and insertion counts using jiwer.	FR-2.5, NFR-1.3
Moodle reading	The Moodle REST web service is called to get the site info, find a course by its short name and find a week section by its number.	FR-5.1
Moodle writing	Playwright logs in to Moodle, creates an assignment with a due date and an attached PDF, uploads material to a week and creates a course, and reports the error text when a save fails.	FR-5.1, FR-5.2, FR-5.3
Agent with tools	A local model served by Ollama is given the Moodle actions as tools and completes a request made in plain words, in a loop that stops after ten steps.	FR-5.1 to FR-5.5
Approval step	Every Moodle action asks the teacher to approve before the browser is opened, and returns a cancelled result if the answer is no.	FR-5.5, NFR-2.1

Structured extraction	An assignment is extracted from free text into a fixed schema with title, course, due date and description using the local model with a JSON schema.	FR-3.6

Appendix B: Traceability of objectives to requirements

Objective	Functional requirements	Use cases
1. Record and process classroom audio	FR-1.1 to FR-1.3	UC-01, UC-02
2. Accurate mixed language transcript with speakers	FR-2.1 to FR-2.5	UC-03, UC-04
3. Academic output from the transcript	FR-3.1 to FR-3.7	UC-05, UC-06
4. Organise the archive and track the outline	FR-4.1 to FR-4.5	UC-01, UC-02
5. Perform the portal and LMS work	FR-5.1 to FR-5.6	UC-07, UC-08, UC-09
6. Deliver on Telegram and WhatsApp	FR-6.1 to FR-6.3	UC-10
7. Index the semester, quizzes and papers	FR-7.1 to FR-8.2	UC-11, UC-12
8. Run offline or on a paid provider	FR-10.1 to FR-10.4, FR-11.1, FR-11.2	UC-05, UC-11

Appendix C: Document revision history

Version	Date	Change	Author
1.0	November 2026	First version written from the approved project proposal.	Asad Saleem, Shoaib Ahmed, Asghar Ali